TypeScriptにおけるXMLのギャップを埋める（ローカライズワークフロー向け）

ロドルフォ・M. ラヤ Maxprograms rmraya@maxprograms.com
2026年6月17日
© 2026 Maxprograms

ローカリゼーションワークフローにおけるXML処理のためにJavaからTypeScriptへの移行について解説し、TMX、XLIFF、SRXのDTDおよびXMLスキーマ検証をNode.jsで実現するために構築されたライブラリ「TypesXML」、「TypesBCP47」、「TypesXLIFF」について説明します。

はじめに

Node.js や TypeScript で XML を処理しようとしたことがある方なら、ある制限に直面したことがあるでしょう。パーシングは簡単ですが、DTD や XML スキーマに対する検証はそう簡単ではありません。

これは、構造や妥当性が必須である TMX、XLIFF、SRX といったローカリゼーション標準を扱う際に、深刻な問題となります。それらが、データが使用可能かどうかを決定づけるからです。

これが私が直面した課題であり、最終的に解決することに決めました。

従来の環境：XML処理にはJava

TMX、XLIFF、SRX、TBXといったローカライゼーション標準は、すべてXMLベースです。20年以上前にこれらの標準を取り扱い始めた当時、Javaが最も実用的な選択肢でした。JavaのXMLツールは完璧ではありませんでしたが、解析や検証を行うには十分な信頼性がありました。

長い間、それはうまく機能していました。

問題はXMLそのものではなく、その周辺環境にありました。

JavaのデスクトップUIの選択肢は、私のニーズに合うように進化しませんでした。私は長年SWTを使用してきましたが、そのメンテナンスは疎かになり、SwingやJavaFXといった代替手段には魅力を感じませんでした。

TypeScriptへの移行

2018年頃、私はユーザーインターフェースの構築をHTML、CSS、JavaScriptへと移行しました。最終的には、静的チェックによるより安全な開発モデルを提供するTypeScriptが、当然の選択肢となりました。

これでUI面の問題は解決しました。

しかし、新たな問題が生じました：

- TypeScript での UI
- XML 処理は依然として Java

単一のワークフローに対して 2 つの技術スタックを維持するのは理想的ではありませんでした。

欠けていたピース：TypeScript での XML 検証

当時、JavaScript には XML パーサーが存在しましたが、以下の機能をサポートしているものは一つもありませんでした：

- DTD による検証
- XML スキーマによる検証

ローカライズ形式においては、これは必須要件です。検証なしでは、TMX や XLIFF ファイルの構造が正しいことを保証できません。

そこで、私は **TypesXML** という、TypeScript 用のネイティブ XML パーサー兼バリデータの開発を始めました。

最小限のサンプル

基本的な XML の解析は以下のようになります：

```
import { Catalog, DOMBuilder, SAXParser } from "typesxml";

const parser: SAXParser = new SAXParser();
const handler: DOMBuilder = new DOMBuilder();
const catalog = new Catalog('/pathTo/catalog.xml');

parser.setCatalog(catalog);
parser.setContentHandler(handler);
parser.setValidating(true);
parser.parseFile("example.xml");

const document = handler.getDocument();
console.log(document.toString());
```

同じパーサーを次のように設定することもできます：

- DTD に基づいて検証を行う
- XML スキーマに基づいた検証

- XMLカタログの解決

これにより、従来Javaライブラリで可能だったのと同等の制御レベルで、Node.js上でXMLを処理することが可能になります。

不足していたスタックの構築

完全なXMLスタックの構築には時間がかかりました。

- 最初にDOMおよびSAXスタイルの解析が実現された
- 続いてDTD検証が行われました
- XMLスキーマの検証が最後のステップでした

XMLスキーマのサポートははるかに複雑ですが、XLIFFのようなフォーマットには不可欠です。

関連要素：言語データ

ローカライゼーションのワークフローも、言語コードに大きく依存しています。

BCP47が標準を定義していますが、公式データは巨大なテキストファイルとして配布されています。TypeScriptで使いやすくするために、私は**TypesBCP47**というコンパニオンライブラリを作成しました。これは、そのデータを解析し、構造化された形で公開するものです。

概念実証：既存ツールの書き換え

このアプローチの有効性を検証するため、既存のツールをTypeScriptで書き直しました：

- **SRXEditor**
- **TMXValidator**

これらのツールは：

- 大規模なXMLファイルを処理し
- 公式の文法に基づいて検証を行う
- 元のJava版と同等のパフォーマンスを維持

これにより、TypeScriptが実環境におけるローカライゼーションのワークロードに対応できることが確認されました。

XLIFF のサポート

XLIFF は XML スキーマを使用しているため、完全なスキーマバリデータの実装が必要でした。

それが整ったところで、私は **TypesXLIFF** という、XLIFF 2.x ファイルの解析、生成、および検証を行うライブラリを作成しました。

以下の機能をサポートしています：

- XML スキーマによる検証
- XLIFF 仕様に対する検証
- 型付きオブジェクトモデル
- JSON 変換

これにより可能になること

これらの要素が整ったことで、以下のことが可能になりました：

- Node.js 上で TMX、XLIFF、SRX を直接検証する
- ローカライゼーションパイプラインを完全に TypeScript で構築する
- 別途 Java バックエンドを用意する必要がなくなる
- UI と処理の両方を同じ言語で統一できる

今後の予定

TBX のサポートを進めているほか、他のツールについても TypeScript へ完全移行するための更新を行っています。

古いコンポーネントにはまだ一部の Java コードが残っていますが、新規開発は現在 TypeScript を中心に進められています。

まとめ

長い間、ローカリゼーションワークフローにおける XML 処理には、事実上 Java が必要とされていました。

しかし、もはやそうではありません。

Node.js で XML ベースのフォーマットを扱う場合、特にローカリゼーションや構造化データパイプラインでは、解析、検証、処理をすべて TypeScript で処理できるようになりました。

著者について



Maxprograms SASのCTOであるRodolfo M. Raya氏は、XLIFF、TMX、TBX、SRX、GlossMLなどのローカリゼーション標準の開発に携わっています。

彼は、このサイトで提供されているソフトウェアや検証サービスを含め、これらの仕様を実装するツールやライブラリを開発しています。